

Today 8:04 AM

The `async/await` pattern relies on a state machine built by the compiler. When execution hits an `await`, the method pauses, hands control back to the caller, and registers a continuation callback. Using `.Result` or `.Wait()` forcefully halts the current thread until the task finishes - explain experienced interview level

At an experienced interview level, don't stop at " `async/await` doesn't block the thread."

Explain **what the compiler generates, what happens at `await`, and why**

`.Result` / `.Wait()` are dangerous.

How `async/await` works internally

When you mark a method `async`, the C# compiler transforms it into a **state machine**. This state machine keeps track of where execution stopped, local variables that must survive the pause, the awaited `Task`, and where execution should continue afterward.

For example:

```
public async Task<string> GetDataAsync()
{
    var result = await httpClient.GetStringAsync(url);
    return result;
}
```

Conceptually, execution works like this:

```
Caller Thread
  |
  v
GetDataAsync()
  |
  v
Start HTTP request
  |
  v
Is Task completed?
```



```

    / \
  Yes  No
  |    |
  |    +--> Save current state
  |          Register continuation
  |          Return Task to caller
  |          Release thread
  |
  v
Continue execution

```

The important point is that `await` **normally does not block the current thread**.

If `GetStringAsync()` hasn't completed, the method saves its state, registers the remainder of the method as a continuation, and returns an incomplete `Task` to its caller.

While the HTTP request is waiting on network I/O, an ASP.NET Core worker thread does **not need to sit there waiting**.

When the I/O operation completes, the task becomes complete and the continuation is scheduled. The state machine resumes from the statement after `await`.

Does `async` create a new thread?

No.

This is one of the most important interview points.

```
</> C#
```



does not mean:

```

Create new thread
  ↓
Execute operation
  ↓
Wait for completion

```



For I/O operations such as database queries, HTTP calls, file I/O, or network calls, the operating system can perform the asynchronous I/O without keeping a .NET worker thread blocked.

So `async/await` is primarily about **non-blocking execution and scalability**, not automatically creating threads or making code execute in parallel.

What changes with `.Result` or `.Wait()`?

Consider this:

```
public string GetData()
{
    var task = httpClient.GetStringAsync(url);

    return task.Result;
}
```

`.Result` says:

Don't return control. Block this thread until the Task finishes.

Similarly:

`</> C#`



blocks the current thread.

The difference is therefore:

```
await
↓
Operation incomplete
↓
Return thread to thread pool
↓
Operation completes
↓
Schedule continuation
↓
Continue execution
```



versus:

```
.Result / .Wait()
↓
Operation incomplete
↓
BLOCK CURRENT THREAD
↓
Thread does nothing useful
↓
```



Operation completes



Thread continues

This is why we commonly say:

`await` is asynchronous waiting; `.Result` and `.Wait()` are synchronous blocking.

Why is blocking especially bad in ASP.NET Core?

Imagine your API receives **1,000 concurrent requests**, and every request calls another API that takes two seconds.

With asynchronous code:

```
public async Task<IActionResult> GetCustomer()
{
    var customer = await customerService.GetCustomerAsync();

    return Ok(customer);
}
```

while requests are waiting for external I/O, ASP.NET Core can return worker threads to the **ThreadPool** and use them to process other requests.

But suppose developers write:

```
public IActionResult GetCustomer()
{
    var customer = customerService
        .GetCustomerAsync()
        .Result;

    return Ok(customer);
}
```

Now each request may occupy a ThreadPool thread while doing nothing except waiting.

Under heavy load:

Many Requests



`.Result` / `.Wait()`



Worker threads blocked



↓
ThreadPool has fewer available threads
↓
New requests wait
↓
Latency increases
↓
ThreadPool starvation
↓
Application throughput drops

This is why the common design principle is:

Async all the way.

If the controller is asynchronous, the service should normally be asynchronous, and the repository/database operations should use their asynchronous APIs too:

```
// Controller
public async Task<IActionResult> GetCustomer(int id)
{
    var customer = await customerService.GetAsync(id);
    return Ok(customer);
}

// Service
public async Task<CustomerDto> GetAsync(int id)
{
    return await customerRepository.GetAsync(id);
}

// Repository
public async Task<Customer?> GetAsync(int id)
{
    return await dbContext.Customers
        .AsNoTracking()
        .FirstOrDefaultAsync(x => x.Id == id);
}
```

What about deadlocks?

This is another strong interview point, but with an important distinction.

In older ASP.NET, WinForms, and WPF applications, blocking with `.Result` or `.Wait()` can cause a classic **SynchronizationContext deadlock**.

For example:

</> C#

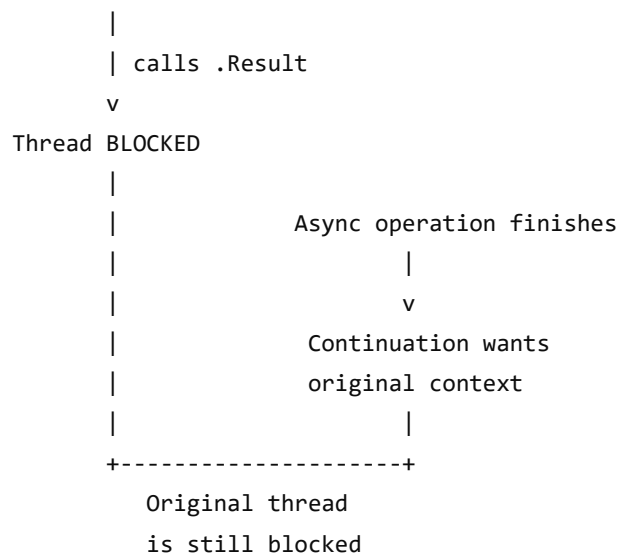


```
public string GetData()
{
    return GetDataAsync().Result;
}

public async Task<string> GetDataAsync()
{
    var result = await SomeOperationAsync();
    return result;
}
```

A simplified scenario is:

UI / Request Thread



Neither side can proceed → **deadlock**.

However, an experienced candidate should add this qualification:

ASP.NET Core does not have the classic ASP.NET `SynchronizationContext`, so the traditional `.Result` / `await` deadlock scenario is much less applicable. But blocking is still harmful because it wastes `ThreadPool` threads and can cause `ThreadPool` starvation and poor scalability.

`ConfigureAwait(false)` — senior-level follow-up

You may also be asked:

</> C#



```
await SomeOperationAsync().ConfigureAwait(false);
```

In environments with a `SynchronizationContext`, this tells the awaiter that the continuation does not need to resume on the captured context.

This is particularly relevant to reusable library code.

In **ASP.NET Core**, there is normally no request `SynchronizationContext` to capture, so `ConfigureAwait(false)` generally doesn't provide the same benefit it historically provided in classic ASP.NET/UI applications.

Interview-ready answer

A strong 60–90 second answer would be:

"`async/await` is implemented by the C# compiler using a state machine. When execution reaches an `await`, it checks whether the awaited task has already completed. If it hasn't, the state machine stores the current execution state, registers a continuation, and returns control to the caller rather than blocking the current thread.

When the asynchronous I/O completes, the task signals completion and the continuation is scheduled, allowing the method to resume from where it stopped.

Async doesn't automatically create a new thread. For I/O-bound operations such as HTTP calls or database access, the main advantage is that we don't keep a `ThreadPool` thread blocked while waiting for I/O.

`.Result` and `.Wait()` are different because they synchronously block the current thread until the task completes. In ASP.NET Core, excessive blocking can consume `ThreadPool` threads, cause `ThreadPool` starvation, increase latency, and reduce application throughput.

In older ASP.NET or UI applications, `.Result` and `.Wait()` can additionally cause `SynchronizationContext` deadlocks when an awaited continuation needs the same context whose thread is currently blocked.

That's why in web applications I generally follow the 'async all the way' approach—from controller to service to repository—and use asynchronous APIs such as `ToListAsync`, `SaveChangesAsync`, and `SendAsync` rather than mixing asynchronous operations with synchronous blocking."

One line to remember for interviews: `await` pauses the method without normally blocking the thread; `.Result` and `.Wait()` pause the method by blocking the thread.

ChatGPT can make mistakes. Check important info.

